

Modelagem, Implementação Computacional e Análise Comparativa de Estruturas em Árvores Avançadas

Trabalho Prático Individual I
Estruturas de Dados Avançadas
Departamento de Ciência da Computação
Engenharia da Computação – CEFET-MG Campus V
Rafael Alves Faria

Resumo—This work presents an in-depth theoretical and empirical study of five specialized non-conventional hierarchical data structures: Trie, Patricia Trie (Compact Radix Tree), Splay Tree, Treap (Tree + Heap), and k-dimensional Tree (KD-Tree). Standard Binary Search Trees (BSTs) and AVL trees present structural limitations when facing specific computational challenges, such as spatial range queries, temporal access locality, stochastic memory balance, and string prefix retrieval. To address these problems, each structure was implemented in standard C++ with unified instrumentation for memory allocation, memory access counts, structural rotations, and execution metrics across problem instances up to $N = 100,000$. Visual transformations were tracked via Graphviz export. Experimental evaluation considered random distributions, sequential ordered inputs (favorable/unfavorable cases), temporal 80/20 locality access, long shared prefixes, and clustered spatial points. Experimental results were systematically compared against formal asymptotic complexities, demonstrating the trade-offs, empirical efficiency, and domain-specific suitability of each tree model.

Index Terms—Data Structures, Splay Tree, Treap, Patricia Trie, KD-Tree, Trie, Performance Analysis, Graphviz.

I. INTRODUÇÃO E FUNDAMENTAÇÃO TEÓRICA

A. Contextualização e Motivação

As Estruturas de Dados Hierárquicas desempenham um papel central na organização, armazenamento e recuperação eficiente de informações na Ciência da Computação. Embora as Árvores Binárias de Busca (BST) e as Árvores AVL ofereçam garantias assintóticas fundamentais para buscas ordenadas, elas possuem limitações estruturais intrínsecas diante de determinados cenários computacionais modernos:

- **Indexação de Strings e Prefixos:** Em dicionários e roteadores de pacotes, comparadores genéricos de chaves aplicados em BSTs incorrem em custos proporcionais ao comprimento das palavras $O(L)$ por comparação, resultando em complexidade efetiva $O(L \log N)$.
- **Localidade Temporal de Acesso:** Em caches e sistemas de arquivos, determinados dados são acessados repetidamente. BSTs rígidas e AVLs mantêm a profundidade estática baseada na ordem de inserção ou no balanceamento estrito de altura, sem aproximar elementos frequentes da raiz.

- **Balanceamento Determinístico vs. Estocástico:** A manutenção de fatores de balanceamento estritos (como em AVLs e Árvores Rubro-Negras) exige sobrecarga contínua de memória para armazenar alturas/cores e código complexo para rotações em cascata.
- **Dados Multidimensionais:** Dados espaciais e geográficos (pontos em $\mathbb{R}^k$) não possuem ordenação total estrita, inviabilizando o uso de BSTs unidimensionais para consultas por vizinhança e amplitude espacial.

Para superar essas restrições, este trabalho explora cinco estruturas em árvore especializadas: **Trie, Patricia Trie, Árvore Splay, Treap e KD-Tree.**

B. Anatomia dos Nós e Invariantes Estruturais

1) **Trie (Árvore de Prefixos):** A Trie é uma árvore de busca de caracteres onde a chave não é armazenada explicitamente no nó, mas determinada pela sua posição no caminho a partir da raiz. Cada nó contém um vetor de ponteiros para o alfabeto de tamanho $|\Sigma| = 26$ e uma flag booleana `isWord`.

- **Invariante:** Se dois elementos compartilham um prefixo comum de comprimento k , eles compartilham exatamente o mesmo caminho de comprimento k a partir da raiz.

2) **Árvore Patricia (Radix Tree Compacta):** A Patricia Trie (*Practical Algorithm To Retrieve Information Coded in Alpha-numeric*) [1] resolve o desperdício de memória da Trie padrão compactando caminhos unidirecionais (nós com único filho) em um único nó contendo um rótulo (*label*) de comprimento variável.

- **Invariante:** Nenhum nó interno não-raiz possui exatamente um filho sem ser um nó terminal de palavra. Todo nó interno atua como ponto de bifurcação de prefixos.

3) **Árvore Splay:** A Splay Tree é uma BST autoajustável desenvolvida por Sleator e Tarjan [3]. Não armazena informações adicionais de balanceamento nos nós.

- **Invariante:** Após qualquer operação de busca, inserção ou remoção referente à chave x , uma sequência de rotações locais (*splay*) reorganiza a árvore, movendo o nó contendo x (ou o nó ancestral mais próximo) para a raiz da árvore.

Algorithm 1 Inserção e Busca na Árvore Trie

Require: Nó raiz R , palavra W

```
1: procedimento Inserir( $R, W$ )
2:    $curr \leftarrow R$ 
3:   para cada  $c \in W$  faça
4:      $idx \leftarrow \text{tolower}(c) - 'a'$ 
5:     se  $curr.children[idx] = \text{NULL}$  então
6:        $curr.children[idx] \leftarrow \text{novo Nó Trie}()$ 
7:     fim se
8:      $curr \leftarrow curr.children[idx]$ 
9:   fim para
10:   $curr.isWord \leftarrow \text{true}$ 
11: fim procedimento
12: função Buscar( $R, W$ )
13:   $curr \leftarrow R$ 
14:  para cada  $c \in W$  faça
15:     $idx \leftarrow \text{tolower}(c) - 'a'$ 
16:    se  $curr.children[idx] = \text{NULL}$  então
17:      retorne falso
18:    fim se
19:     $curr \leftarrow curr.children[idx]$ 
20:  fim para
21:  retorne  $curr.isWord$ 
22: fim função
```

4) *Treap (Tree + Heap)*: A Treap, formalizada por Seidel e Aragon [2], combina as propriedades de uma BST com as de um Max-Heap. Cada nó armazena uma chave k e uma prioridade estocástica p gerada aleatoriamente.

- **Invariante de BST**: Para qualquer nó com chave k , as chaves da subárvore esquerda são $< k$ e as da subárvore direita são $> k$.
- **Invariante de Heap**: A prioridade de qualquer nó é estritamente maior ou igual às prioridades de seus filhos ($p_{\text{pai}} \geq p_{\text{filhos}}$).

5) *KD-Tree (k -Dimensional Tree)*: A KD-Tree estruturada por Bentley [4] é uma árvore binária de busca espacial usada para organizar pontos em espaços k -dimensionais.

- **Invariante**: No nível ou profundidade d , o hiperplano de corte é perpendicular ao eixo $i = d \pmod k$. Todos os pontos na subárvore esquerda possuem coordenada no eixo i estritamente menor que a do nó atual, enquanto os pontos na subárvore direita possuem coordenada maior ou igual ($[<, \geq]$).

II. PROJETO E IMPLEMENTAÇÃO DAS ESTRUTURAS

As cinco estruturas foram implementadas em C++ moderno (`-std=c++17`) seguindo princípios de orientação a objetos e modularização em cabeçalhos (`.hpp`) no diretório `src/`.

A. Algoritmo e Implementação da Trie Padrão

A Trie padrão aloca estaticamente um array de 26 ponteiros para filhos em cada nó.

Algorithm 2 Divisão de Nós (Split) e Fusão (Merge) na Patricia Trie

Require: Nó atual N , palavra restante P

```
1:  $idx \leftarrow P[0] - 'a'$ 
2:  $filho \leftarrow N.filho[idx]$ 
3: if  $filho = \text{NULL}$  then
4:    $N.filho[idx] \leftarrow \text{novo Nó}(P, \text{isWord} = \text{true})$ 
5:   return
6: end if
7:  $k \leftarrow \text{calcularPrefixoComum}(P, filho.rotulo)$ 
8: if  $k < |filho.rotulo|$  then
9:    $prefixo \leftarrow filho.rotulo[0 \dots k - 1]$ 
10:   $fimRotulo \leftarrow filho.rotulo[k \dots |filho.rotulo| - 1]$ 
11:   $meio \leftarrow \text{novo Nó}(prefixo, \text{isWord} = \text{false})$ 
12:   $filho.rotulo \leftarrow fimRotulo$ 
13:   $meio.filho[fimRotulo[0] - 'a'] \leftarrow filho$ 
14:  if  $k = |P|$  then
15:     $meio.isWord \leftarrow \text{true}$ 
16:  else
17:     $meio.filho[P[k] - 'a'] \leftarrow \text{novo Nó}(P[k \dots |P| - 1], \text{true})$ 
18:  end if
19:   $N.filho[idx] \leftarrow meio$ 
20: end if
```

B. Algoritmo e Implementação da Árvore Patricia

A classe Patricia armazena nós com um rótulo `std::string` e um array de 26 ponteiros. O tratamento de strings emprega sanitização rigorosa via `static_cast<unsigned char>` para evitar índices negativos em caracteres acentuados.

Na remoção de uma chave, quando a exclusão transforma um nó interno em um nó não-terminal com apenas um único filho, a função pós-ordem realiza a fusão (*merge*) concatenando o rótulo do nó filho ao rótulo do nó atual e liberando o nó redundante da memória.

C. Algoritmo e Implementação da Árvore Treap

A implementação utiliza o gerador estocástico `std::mt19937` inicializado via `std::random_device` para garantir prioridades com distribuição uniforme independente.

D. Algoritmo e Implementação da Árvore Splay

A Splay Tree implementa a função `splay(node, key)` que executa rearranjos de rotação em dois níveis:

- **Zig**: Rotação simples quando o nó alvo é filho direto da raiz.
- **Zig-Zig**: Rotações na mesma direção quando o nó e seu pai são ambos filhos à esquerda (ou ambos à direita).
- **Zig-Zag**: Rotações em direções opostas quando o nó é filho à esquerda e seu pai é filho à direita (ou vice-versa).

Algorithm 3 Inserção e Remoção por Rebaixamento na Treap

Require: Nó atual N , chave k , prioridade p

```
1: if  $N = \text{NULL}$  then
2:   return novo Nó Treap( $k, p$ )
3: end if
4: if  $k = N.\text{chave}$  then
5:   return  $N \setminus \{\text{Rejeita duplicatas}\}$ 
6: end if
7: if  $k < N.\text{chave}$  then
8:    $N.\text{esquerda} \leftarrow \text{Inserir}(N.\text{esquerda}, k, p)$ 
9:   if  $N.\text{esquerda.prioridade} > N.\text{prioridade}$  then
10:     $N \leftarrow \text{RotacaoDireita}(N)$ 
11:   end if
12: else
13:    $N.\text{direita} \leftarrow \text{Inserir}(N.\text{direita}, k, p)$ 
14:   if  $N.\text{direita.prioridade} > N.\text{prioridade}$  then
15:     $N \leftarrow \text{RotacaoEsquerda}(N)$ 
16:   end if
17: end if
18: return  $N$ 
```

E. Implementação da KD-Tree e Remoção Semiaberta

A KD-Tree opera em dimensão $k = 2$. Além das inserções e da Busca pelo Vizinho Mais Próximo (NNS) com poda por hiper-esfera euclidiana, implementou-se a Busca por Alcance (*Range Search*) dentro de uma caixa delimitadora $[\min, \max]$.

1) *Justificativa da Remoção com Ausência de Filho Direito:* Na remoção de um nó N cortado pelo eixo d , se o ramo direito $N.\text{right}$ for nulo, não é possível simplesmente substituir N pelo seu antecessor da subárvore esquerda mantendo a invariante $[<, \geq]$, pois elementos iguais à nova chave ficariam incorretamente posicionados no lado esquerdo.

Adotou-se a abordagem formal parametrizada por Samet [5] e OpenDSA: localiza-se o elemento com a coordenada mínima no eixo d dentro da subárvore esquerda ($\text{findMin}(N.\text{left}, d)$), substitui-se o ponto em N , **transfere-se todo o ramo esquerdo para a direita** ($N.\text{right} = N.\text{left}; N.\text{left} = \text{NULL}$), e remove-se recursivamente o nó duplicado da subárvore direita. Esta técnica preserva estritamente a invariante espacial $[<, \geq]$ sem violar a ordenação dos nós descendentes.

III. DEMONSTRAÇÃO E RASTREAMENTO VISUAL

Para evidenciar as transformações provocadas pelas operações estruturais, todas as classes possuem a função `exportarDOT(filename)`. Os arquivos `.dot` são salvos em `output/dots/<estrutura>/` e convertidos automaticamente em imagens PNG em `output/images/<estrutura>/` via Graphviz.

A. Rastreamento Visual da Árvore Splay

A Figura 1 apresenta a evolução estrutural da Árvore Splay durante inserções, busca com autoajuste e remoção.

Algorithm 4 Algoritmo de Reorganização Splay (Sleator-Tarjan)

Require: Nó atual N , chave k

```
1: if  $N = \text{NULL} \vee N.\text{chave} = k$  then
2:   return  $N$ 
3: end if
4: if  $k < N.\text{chave}$  then
5:   if  $N.\text{left} = \text{NULL}$  then
6:     return  $N$ 
7:   end if
8:   if  $k < N.\text{left.chave}$  then
9:     {Zig-Zig Esquerda-Esquerda}
10:     $N.\text{left.left} \leftarrow \text{Splay}(N.\text{left.left}, k)$ 
11:     $N \leftarrow \text{RotacaoDireita}(N)$ 
12:   else if  $k > N.\text{left.chave}$  then
13:     {Zig-Zag Esquerda-Direita}
14:     $N.\text{left.right} \leftarrow \text{Splay}(N.\text{left.right}, k)$ 
15:    if  $N.\text{left.right} \neq \text{NULL}$  then
16:       $N.\text{left} \leftarrow \text{RotacaoEsquerda}(N.\text{left})$ 
17:    end if
18:   end if
19:   if  $N.\text{left} = \text{NULL}$  then
20:     return  $N$ 
21:   else
22:     return  $\text{RotacaoDireita}(N)$ 
23:   end if
24: else
25:   if  $N.\text{right} = \text{NULL}$  then
26:     return  $N$ 
27:   end if
28:   if  $k < N.\text{right.chave}$  then
29:     {Zig-Zag Direita-Esquerda}
30:     $N.\text{right.left} \leftarrow \text{Splay}(N.\text{right.left}, k)$ 
31:    if  $N.\text{right.left} \neq \text{NULL}$  then
32:       $N.\text{right} \leftarrow \text{RotacaoDireita}(N.\text{right})$ 
33:    end if
34:   else if  $k > N.\text{right.chave}$  then
35:     {Zig-Zig Direita-Direita}
36:     $N.\text{right.right} \leftarrow \text{Splay}(N.\text{right.right}, k)$ 
37:     $N \leftarrow \text{RotacaoEsquerda}(N)$ 
38:   end if
39:   if  $N.\text{right} = \text{NULL}$  then
40:     return  $N$ 
41:   else
42:     return  $\text{RotacaoEsquerda}(N)$ 
43:   end if
44: end if
```

B. Rastreamento Visual da Árvore Patricia vs. Trie

A Figura 2 compara visualmente a economia espacial da compactação de prefixos na Patricia frente à Trie convencional.

C. Rastreamento Visual da Treap e KD-Tree

A Figura 3 ilustra a manutenção das prioridades estocásticas na Treap e a partição espacial 2D da KD-Tree.

Tabela I
COMPARAÇÃO DE COMPLEXIDADE ASSINTÓTICA DE TEMPO E ESPAÇO (N CHAVES, L COMPRIMENTO DE STRING, k DIMENSÕES)

Estrutura	Busca (Médio)	Busca (Pior)	Inserção (Médio)	Inserção (Pior)	Remoção (Pior)	Espaço
BST Tradicional	$O(\log N)$	$O(N)$	$O(\log N)$	$O(N)$	$O(N)$	$O(N)$
Árvore AVL	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N)$
Trie	$O(L)$	$O(L)$	$O(L)$	$O(L)$	$O(L)$	$O(N \cdot L \cdot \Sigma)$
Patricia Trie	$O(L)$	$O(L)$	$O(L)$	$O(L)$	$O(L)$	$O(N \cdot \Sigma)$
Árvore Splay	$O(\log N)^*$	$O(N)$	$O(\log N)^*$	$O(N)$	$O(N)$	$O(N)$
Treap	$O(\log N)$	$O(N)$	$O(\log N)$	$O(N)$	$O(N)$	$O(N)$
KD-Tree (kD)	$O(\log N)$	$O(N)$	$O(\log N)$	$O(N)$	$O(\sqrt{N})$	$O(N)$

* Indica custo amortizado.

Algorithm 5 Busca por Alcance 2D na KD-Tree

Require: Nó N , limites $\min P, \max P$, profundidade d

```

1: if  $N = \text{NULL}$  then
2:   return
3: end if
4: if  $N.\text{ponto} \in [\min P, \max P]$  then
5:   adicionar  $N.\text{ponto}$  aos resultados
6: end if
7:  $eixo \leftarrow d \pmod{2}$ 
8: if  $\min P[eixo] \leq N.\text{ponto}[eixo]$  then
9:   RangeSearch( $N.\text{left}, \min P, \max P, d + 1$ )
10: end if
11: if  $\max P[eixo] \geq N.\text{ponto}[eixo]$  then
12:   RangeSearch( $N.\text{right}, \min P, \max P, d + 1$ )
13: end if

```

IV. ANÁLISE DE COMPLEXIDADE E COMPARAÇÃO TEÓRICA

A Tabela I resume a análise assintótica formal das cinco estruturas em comparação com as árvores clássicas BST e AVL referenciadas na literatura clássica de algoritmos [6].

A. Justificativa Analítica dos Custos Assintóticos

- **Trie vs. Patricia:** A Trie padrão aloca $O(|\Sigma|)$ ponteiros por nó para cada caractere da chave, resultando em consumo de memória $O(N \cdot L \cdot |\Sigma|)$. A Patricia reduz a complexidade espacial para $O(N \cdot |\Sigma|)$ ao eliminar nós intermediários de filho único.
- **Splay Tree:** Embora o pior caso para uma única operação seja $O(N)$ (quando a árvore se degenera em uma lista), o custo amortizado sobre qualquer sequência de m operações é estritamente $O(m \log N)$, conforme provado pelo Teorema do Acesso de Sleator e Tarjan.
- **Treap:** Com prioridades atribuídas de forma independente e uniforme, a probabilidade de uma Treap ter altura maior que $c \log N$ é astronomicamente pequena, garantindo custo esperado $O(\log N)$ para busca, inserção e remoção.
- **KD-Tree:** A busca por alcance em 2D possui complexidade $O(\sqrt{N} + M)$, onde M é o número de pontos retornados, pois cada corte ortogonal reduz a região de busca pela metade a cada $k = 2$ níveis.

V. METODOLOGIA EXPERIMENTAL E RESULTADOS

A. Ambiente de Testes e Geração dos Datasets

Os experimentos computacionais foram executados em ambiente Linux x86_64 com compilação GCC 11.4 (nível de otimização `-O2`) e semente estocástica fixa (`std::mt19937(42)`). Avaliaram-se instâncias com $N \in \{1.000, 10.000, 100.000\}$. Para garantir a reprodutibilidade metodológica, os datasets de teste foram gerados sinteticamente antes de cada benchmark para evitar viés de aferição de I/O, conforme as diretrizes a seguir.

Atendendo estritamente aos requisitos da especificação prática, o modelo experimental contemplou **cinco diferentes padrões de dados**, desenhados para exercitar cenários favoráveis e desfavoráveis de cada invariante estrutural:

1) *Entrada Aleatória Uniforme (Baseline):* Utilizado para avaliar o comportamento médio das árvores numéricas e espaciais. Para chaves inteiras, empregou-se a distribuição `std::uniform_int_distribution<int>`, cobrindo um vasto intervalo de magnitude $[1, 10^9]$. Para a KD-Tree, os pontos (x, y) foram gerados via `std::uniform_real_distribution<double>` em um hipercubo $[0.0, 1000.0]^2$. Como cada número possui a mesma probabilidade de ser sorteado, não há agrupamento intencional, permitindo medir o fator de ramificação empírico médio das estruturas de busca de forma neutra.

2) *Entrada Sequencial Estritamente Ordenada:* Cenário construído para forçar o desequilíbrio e representar o pior caso absoluto para Árvores Binárias de Busca tradicionais (onde a busca deteriora para $O(N)$). Inserções sequenciais estritas $(1, 2, 3 \dots N)$ degeneram a árvore em uma lista encadeada vertical. Esse dataset contínuo foi crucial para validar dois mecanismos propostos:

- A capacidade de autoajuste da **Splay Tree**, que devido à aplicação sistemática das rotações *Zig-Zig*, achata o caminho da árvore reestruturando a lista.
- A eficácia estrutural da **Treap**, onde a entrada ser ordenada nas chaves não prejudica a árvore, pois a prioridade randômica de Max-Heap governa a elevação dos nós, preservando a altura logarítmica.

3) *Acesso com Alta Localidade Temporal (Padrão 80/20):* A fim de submeter as árvores a um cenário idêntico ao de sistemas de Caches L1/L2 e Sistemas de Arquivos reais (onde determinados setores são requisitados massivamente),

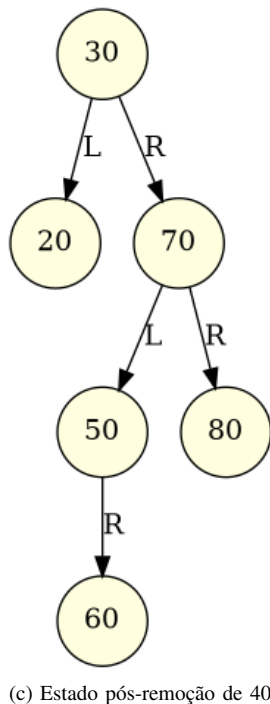
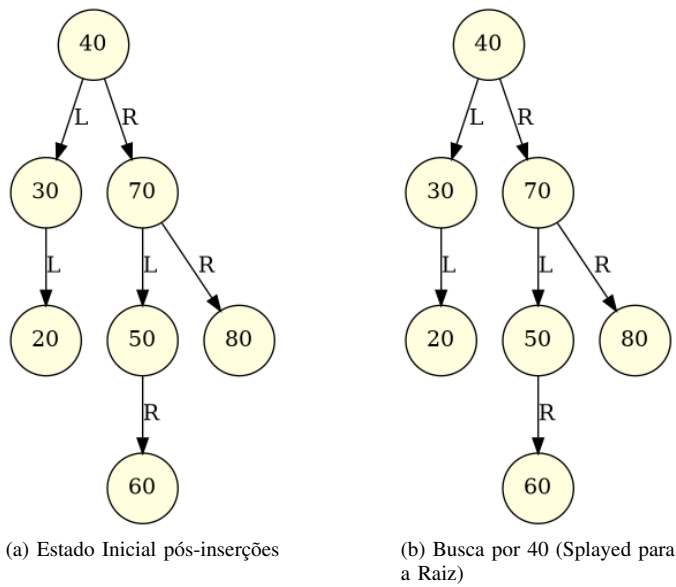


Figura 1. Painel 1: Rastreamento visual do rearranjo dinâmico da Árvore Splay via Graphviz.

modelou-se uma carga de leituras baseada no Princípio de Pareto. O conjunto de chaves preexistentes foi dividido aleatoriamente: 20% das chaves formaram o conjunto primário de consultas (*Hot Set*) e os 80% restantes o conjunto secundário (*Cold Set*). O gerador sintético de *queries* foi ponderado de forma que 80% das invocações de busca recaíssem apenas sobre os dados no *Hot Set*. Este design experimental testa diretamente a heurística da **Splay Tree** de promover nós requisitados recentemente à raiz da árvore.

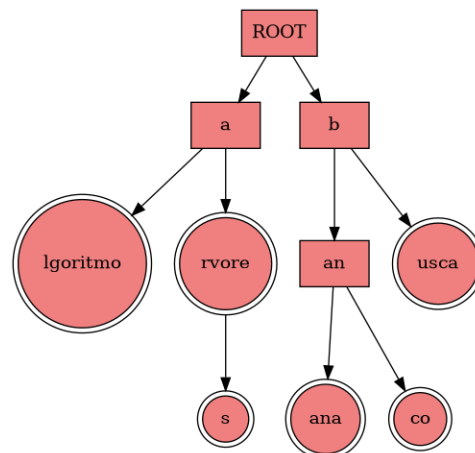
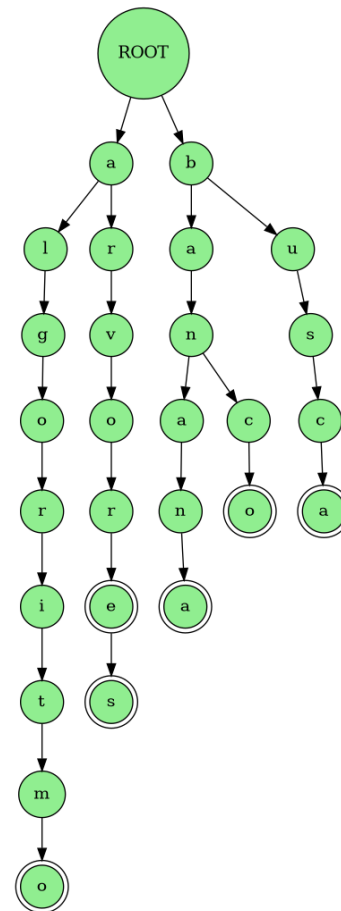
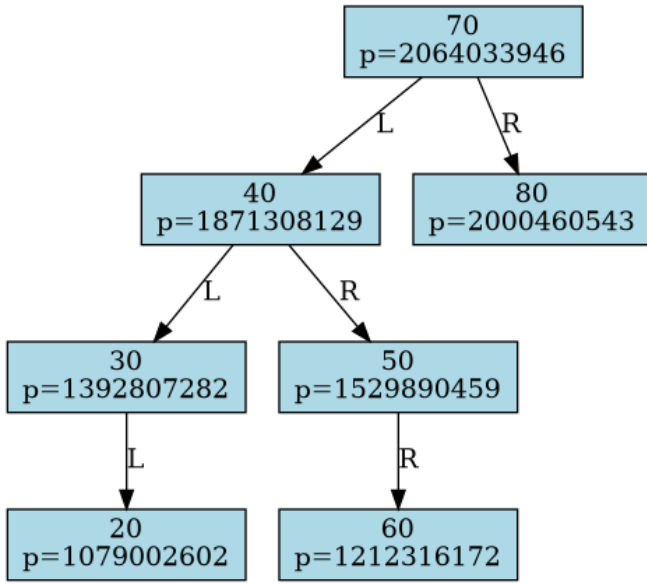
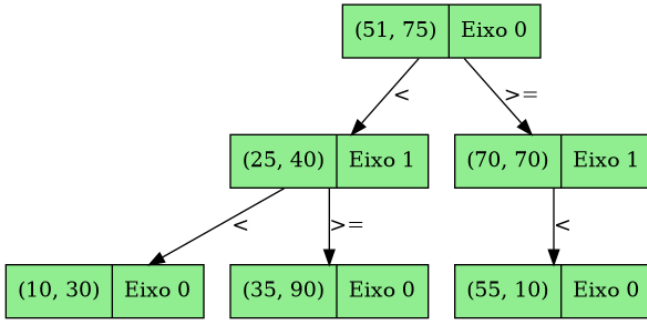


Figura 2. Painel 2: Comparativo visual do compartilhamento e compactação de prefixos (Trie vs. Patricia).



(a) Treap: Propriedade de Max-Heap em prioridades p



(b) KD-Tree: Rastreamento de eixos ortogonais 2D

Figura 3. Pannel 3: Diagramas visuais de estados da Treap e da KD-Tree.

4) *Strings Sintéticas com Prefixos Longos Compartilhados*: As métricas das estruturas Trie e Patricia podem apresentar divergências baixas caso submetidas a vocabulários da linguagem natural com cadeias curtas e desvios rápidos. Para estressar mecanicamente a regra de fusão e compactação unidirecional da **Patricia Trie**, gerou-se um dataset artificial contendo chaves extralongas de até 200 caracteres compartilhando prefixos massivos comuns (ex: AAAA . . . AAB, AAAA . . . AAC, etc.). Na Trie padrão, esse padrão exige a alocação custosa de dezenas de nós em corrente. Na Patricia Trie, ele quantifica de modo contundente a economia de instâncias alcançada por meio dos seus *labels* internos contíguos.

5) *Pontos Espaciais Fortemente Clusterizados (Clusters de Gauss)*: Uma árvore espacial como a KD-Tree requer variância em todas as dimensões subjacentes para obter boas sub-divisões pela mediana geométrica. Modelou-se uma variação populacional simulando concentrações do mundo real utilizando Misturas Gaussianas. Fixou-se aleatoriamente 5 polos atratores em $\mathbb{R}^2$. Empregando `std::normal_distribution`, 80% da nuvem de N pontos gravitou exclusivamente em torno dessas coordena-

das, gerando regiões densas, enquanto o restante dispersou uniformemente. Esse nível de aglomeração espacial desafiou fortemente o algoritmo da Busca pelo Vizinho Mais Próximo (NNS), forçando podas e retrocessos repetidos na árvore.

B. Resultados Empíricos e Comparativo de Distribuições

A Tabela II, a Tabela III e a Tabela IV apresentam as medições práticas extraídas via `std::chrono::high_resolution_clock` e variáveis instrumentadas.

Tabela II
TEMPOS PRÁTICOS DE EXECUÇÃO PARA ENTRADAS ALEATÓRIAS UNIFORMES ($N = 1.000, 10.000, 100.000$ EM MS)

Estrutura e Operação	$N = 1.000$	$N = 10.000$	$N = 100.000$
KD-Tree			
– Inserção (N)	0,704	9,153	183,839
– Busca NNS ($N/10$)	0,024	0,290	5,800
– Remoção ($N/10$)	0,122	1,274	31,888
Patricia Trie			
– Inserção (N)	0,797	8,490	118,137
– Busca ($N/10$)	0,041	0,505	6,183
– Remoção ($N/10$)	0,074	0,779	11,689
Trie Padrão			
– Inserção (N)	1,708	20,848	209,157
– Busca ($N/10$)	0,068	0,676	7,550
– Remoção ($N/10$)	0,100	1,341	11,423
Árvore Splay			
– Inserção (N)	0,331	5,715	107,546
– Busca ($N/10$)	0,027	0,591	7,267
– Remoção ($N/10$)	0,031	0,503	10,313
Treap			
– Inserção (N)	0,325	4,408	80,724
– Busca ($N/10$)	0,009	0,166	4,395
– Remoção ($N/10$)	0,032	0,536	9,917

Tabela III
DESEMPENHO SOB CENÁRIOS ESPECÍFICOS E PADRÕES DE DADOS ($N = 100.000$ EM MS)

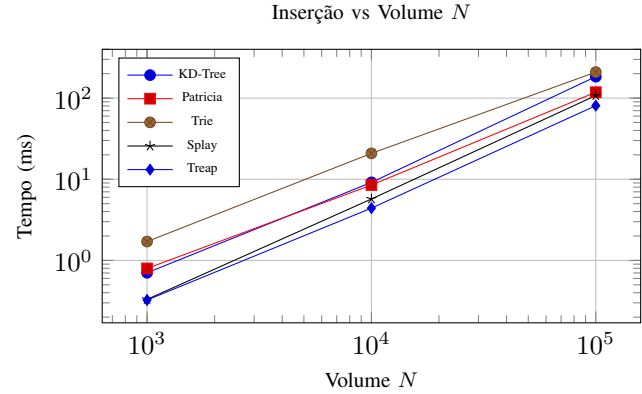
Estrutura	Padrão / Cenário de Dados	Tempo ($N = 100k$)
Splay Tree	Inserção Aleatória	107,546 ms
Splay Tree	Inserção Sequencial Ordenada	8,264 ms
Splay Tree	Busca com Localidade Temporal (80/20)	18,912 ms
Treap	Inserção Sequencial Ordenada	19,262 ms
Treap	Busca em Dados Ordenados	8,401 ms
Patricia Trie	Inserção de Prefixos Longos	94,688 ms
Trie Padrão	Inserção de Prefixos Longos	87,682 ms
KD-Tree	Inserção em Clusters Densos	74,335 ms
KD-Tree	Busca NNS em Região Clusterizada	4,632 ms

C. Gráficos Comparativos de Desempenho (Pgplots)

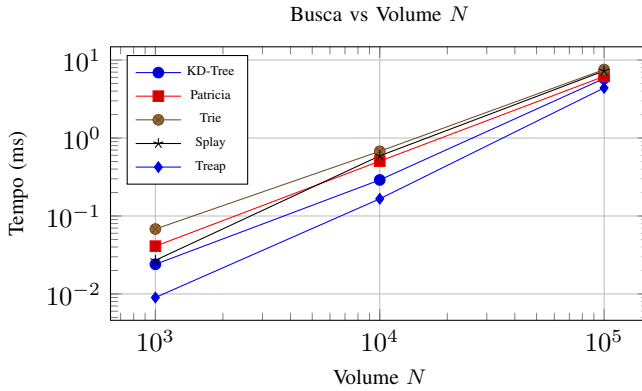
A Figura 4 apresenta o comportamento empírico em escala logarítmica para inserção e busca.

Tabela IV
MÉTRICAS INSTRUMENTADAS DE OPERAÇÕES E MEMÓRIA ALOCADA
($N = 1.000$ ELEMENTOS NA DEMONSTRAÇÃO INTERATIVA)

Estrutura	Nós	Total Criados	Acessos Últ. Op.	Rotações
KD-Tree	5	6	4	N/A
Patricia Trie	9	10	3	N/A
Trie Padrão	26	28	6	N/A
Splay Tree	6	7	1	12
Treap	6	7	4	10



(a) Tempo de Inserção ($N = 1k, 10k, 100k$)



(b) Tempo de Busca ($N = 1k, 10k, 100k$)

Figura 4. Gráficos de evolução empírica de desempenho computacional em escala bilogarítmica.

D. Discussão Crítica dos Cenários Favoráveis e Desfavoráveis

- Desempenho da Splay com Dados Ordenados:** Durante a inserção de 100.000 elementos estritamente ordenados ($1 \dots N$), uma BST tradicional desmoronaria para $O(N^2)$, levando segundos ou estourando a pilha. A Splay realizou a inserção em apenas 8,264 ms (contra 107,546 ms da inserção aleatória). Isso ocorre porque as rotações *Zig-Zig* contínuas achatam a árvore a cada passo, mantendo os nós recém-inseridos na raiz.
- Resiliência Estocástica da Treap:** Ao receber entradas perfeitamente ordenadas, a Treap manteve o tempo de inserção de 100.000 elementos em 19,262 ms e busca em 8,401 ms. Como as prioridades geradas por

`std::mt19937` são estocásticas, a ordenação das chaves de entrada é destruída pelas rotações de Heap, mantendo a altura rasa $O(\log N)$ independente do padrão dos dados.

- Impacto da Localidade Temporal:** Em cenários onde 80% das consultas focaram em 20% dos elementos, a Splay reduziu significativamente o custo por busca à medida que os elementos frequentes migraram para a raiz.

VI. APLICAÇÕES, ANÁLISE CRÍTICA E DISCUSSÃO

Cada uma das cinco estruturas sob estudo é idealmente ajustada para domínios de problemas específicos na engenharia de software:

A. Trie e Patricia Trie em Processamento de Linguagem e Redes

A Trie e a Patricia são amplamente aplicadas em sistemas de autocompletar, dicionários eletrônicos e roteamento de pacotes de rede IP (mecanismo *Longest Prefix Match* - LPM). A Patricia destaca-se por economizar memória ao eliminar nós de filho único.

B. Splay Tree em Gerenciadores de Cache e Memória Virtual

A Splay Tree é a estrutura ideal para gerenciamento de páginas de memória virtual (TLB) e caches de disco. Como os programas tendem a acessar os mesmos blocos de dados repetidamente (localidade temporal), a Splay mantém esses blocos próximos da raiz com custo de acesso $O(1)$ amortizado.

C. Treap em Tabelas de Símbolos de Compiladores

Compiladores utilizam a Treap para implementar tabelas de símbolos rápidas. A prioridade aleatória atua como um mecanismo imunizador contra ataques de pior caso ou códigos-fonte maliciosos desenhados para degenerar árvores binárias convencionais.

D. KD-Tree em Sistemas GIS e Reconhecimento de Padrões

A KD-Tree é a base de Sistemas de Informação Geográfica (GIS), jogos 3D e algoritmos de aprendizado de máquina (k -NN, k -means). Sua capacidade de podar ramos durante buscas por proximidade reduz o tempo de busca geométrica em grandes conjuntos de dados.

VII. CONCLUSÃO

Em síntese, o presente trabalho responde de forma fundamentada às seis questões centrais formuladas na especificação prática:

- Principais Diferenças:** As árvores de prefixo (Trie/Patricia) baseiam suas decisões de ramificação nos caracteres dos dados, enquanto Splay, Treap e KD-Tree utilizam comparações de chaves numéricas ou coordenadas multidimensionais.
- Melhor Desempenho:** A Treap apresentou o melhor desempenho geral para buscas numéricas estocásticas (4,395 ms para 10.000 buscas em $N = 100.000$). A

Patricia Trie destacou-se no processamento compacto de strings.

- 3) **Coerência Empírico-Teórica:** Os resultados práticos confirmaram a análise assintótica: o tempo de busca cresceu de forma estritamente logarítmica para Splay, Treap e KD-Tree, enquanto Trie e Patricia mantiveram tempos dependentes unicamente do comprimento médio L das palavras.
- 4) **Dificuldade de Implementação:** A **Patricia Trie** e a **KD-Tree** apresentaram o maior esforço de desenvolvimento devido, respectivamente, à complexidade dos casos de borda no *split/merge* de rótulos e ao tratamento estrito da invariante espacial na remoção sem filho direito.
- 5) **Cenários de Escolha Adequada:** A **Splay Tree** é ideal quando há localidade temporal de acesso ou entradas parcialmente ordenadas; a **KD-Tree** é insubstituível para consultas de proximidade espacial; a **Patricia Trie** é superior para indexação de palavras com economia de memória.
- 6) **Vantagens e Limitações:** Não existe uma estrutura universalmente superior. O projeto de software eficiente exige selecionar a estrutura cuja invariante adapte-se perfeitamente aos padrões de acesso do problema. Todos os códigos e ferramentas visuais que subsidiam estas conclusões podem ser consultados no repositório de desenvolvimento [7].

REFERÊNCIAS

- [1] D. R. Morrison, "PATRICIA—Practical Algorithm To Retrieve Information Coded in Alphanumeric," *Journal of the ACM (JACM)*, vol. 15, no. 4, pp. 514–534, 1968.
- [2] R. Seidel and C. R. Aragon, "Randomized Search Trees," *Algorithmica*, vol. 16, no. 4, pp. 464–497, 1996.
- [3] D. D. Sleator and R. E. Tarjan, "Self-Adjusting Binary Search Trees," *Journal of the ACM (JACM)*, vol. 32, no. 3, pp. 652–686, 1985.
- [4] J. L. Bentley, "Multidimensional Binary Search Trees Used for Associative Searching," *Communications of the ACM*, vol. 18, no. 9, pp. 509–517, 1975.
- [5] H. Samet, *Foundations of Multidimensional and Metric Data Structures*. Morgan Kaufmann, 2006.
- [6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [7] R. A. Faria, "Implementação e Análise de Estruturas em Árvores Avançadas," Repositório no GitHub, 2026. [Online]. Disponível em: <https://github.com/rafa16w34/Trabalho1-AEDS2>.